Graph representation — how the subgraph reaches the writer

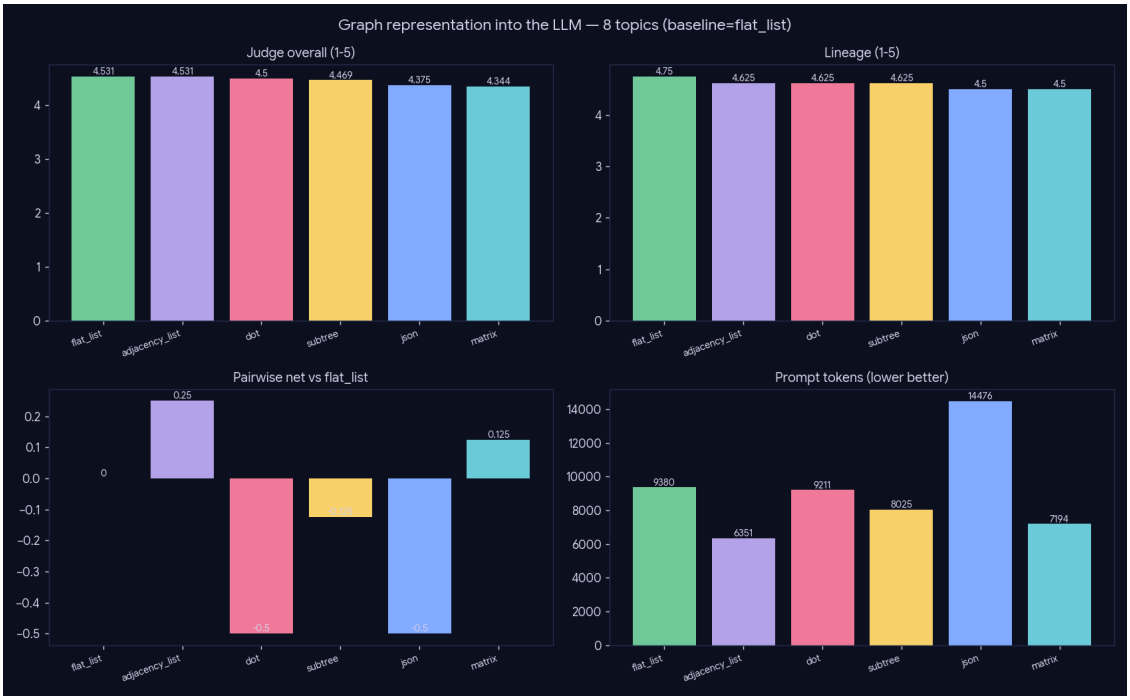
arxivkg · generation · graph representation

What we did

A retrieved citation subgraph (nodes, typed edges, non-linear) has to be serialized into a linear LLM prompt. This is not a solved problem; each encoding has pros/cons. **Phase 1** compares the *structure* axis — `flat_list`, `adjacency_list`, `dot` (Graphviz), `subtree`, `json`, `matrix` — judged absolute (4-axis) and order-debiased pairwise vs `flat_list`, over 8 deep-panel topics at T= 0. **Phase 2** fixes the adjacency winner and layers the *orthogonal* axes: per-node `roles` (foundational/core/frontier), per-node `extracted` content (full-text schema fields vs the abstract), and `alias` (short `[D1]..` labels + an id table).

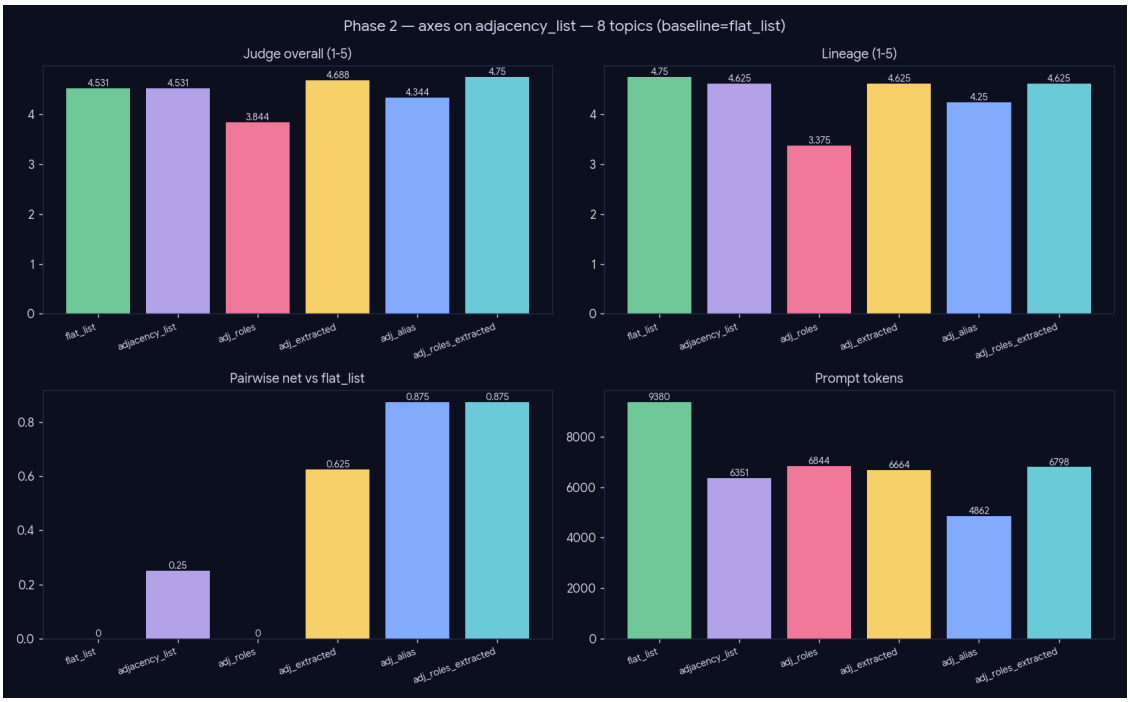
Phase 1 — structure

representation	overall	pair_net	tokens
flat_list	4.312	0.0	9380
adjacency_list	4.531	0.5	6351
dot	4.344	0.25	9211
subtree	4.344	0.375	8025
json	4.469	0.375	14476
matrix	4.438	-0.125	7194



Phase 2 — orthogonal axes on the adjacency winner

config	cov.	lineage	overall	pair_net	tokens
flat_list	3.5	4.5	4.312	0.0	9380
adjacency_list	3.75	4.625	4.531	0.5	6351
adj_roles	4.125	3.875	4.438	0.25	6844
adj_extracted	4.375	4.875	4.719	0.75	6690
adj_alias	4.0	4.75	4.656	0.875	4862
adj_roles_extracted	4.625	5.0	4.906	0.875	6824



Conclusion

Structure: `adjacency_list` **wins** (pair_net 0.5) — and does it at the *fewest* tokens (6351 vs 9380 for flat, 2.3× for json) — a node table + intent-grouped adjacency reads cleanly and compresses. `matrix` loses (negative net); `dot`/ `json` are middling and dearer. **Content is the biggest lever**: `adj_extracted` (full-text fields instead of abstracts) jumps to pair_net 0.75 — specifics the abstract omits are what the composer needs. **Roles help only *with* content**: `adj_roles` alone is a weak 0.25, but combined the winner `adj_roles_extracted` tops every axis (overall 4.906, pairwise 7/1/0). **Aliasing is a free token win**: `adj_alias` matches the top pairwise at the *fewest* tokens (4862). **Production digest: adjacency-list + extracted content + role tags + aliasing. Caveat:** n= 8, absolute axes compressed; the pairwise nets are the discriminator and they agree across both phases.